CoopInfer: Host–Device Cooperative Inference Scheduling

A strategy-driven prototype for DAG partitioning and schedule visualization

CoopInfer Project

June 25, 2026

Motivation

- Embodied AI pipelines often run as DAGs of perception, fusion, and control operators.
- Some operators must stay on the robot-side device, while others may run on an edge host.
- The scheduling problem is not only graph partitioning:
  - cross-device edges incur network transfer cost;
  - each side has a finite execution queue;
  - end-to-end latency constraints can make otherwise low-loss solutions infeasible.
- CoopInfer is a desktop prototype for editing, solving, and visualizing these schedules.

System Overview

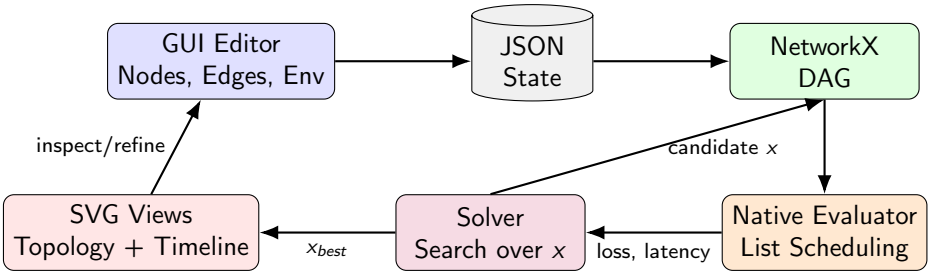
Inputs

- DAG nodes with device/host compute costs.
- Edges with transfer sizes.
- Bandwidth, latency, and objective weights.
- Serialized network model with optional transfer batching.
- Optional average E2E and max-frame latency limits.

Outputs

- Node placement: $x_i = 0$ for DEVICE, $x_i = 1$ for HOST.
- End-to-end latency and device utilization.
- SVG topology and profiler-style schedule timeline.

Prototype Loop

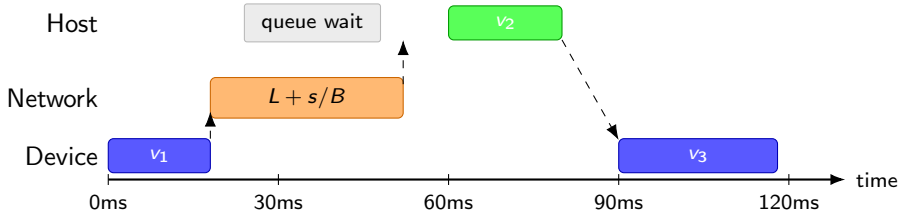


The prototype keeps editing, solving, physical evaluation, and visualization in one reproducible workflow.

Data Model

- In memory, topology is stored in a `networkx.DiGraph`.
- Node attributes:
  - `id`, `name`, `c_dev`, `c_host`, `fixed_dev`, `x`.
  - Optional input cadence fields: `source_period_ms`, `source_phase_ms`.
- Edge attributes:
  - `source`, `target`, `size` in MB.
- Environment fields:
  - `bandwidth`, `latency`, `weight_avg_latency`, `weight_max_latency`, `weight_device_utilization`.
  - `latency_limit`, `max_frame_latency_limit`, `batch_transfers`, `pipeline_unroll`.
- JSON load/save keeps experiments reproducible and shareable.

Queue-Aware Execution



A node starts only after all data is ready and its assigned worker queue is available.

Evaluator Step 1: Expanded Task DAG

For a candidate assignment x, the C++ core expands the unrolled pipeline into a task DAG.

- Source tasks: zero-duration releases at $phase_i + f \cdot period_i$.
- Compute tasks: one task per non-source node and frame.
- Cross-device edges: explicit Network tasks with cost $L + s/B$.
- Batched outgoing transfers: one Network task with $L + \sum_k s_k/B$.

Dependency classes:

- Same-device data edge: source compute/source task $\rightarrow$ target compute task.
- Cross-device data edge: source task $\rightarrow$ Network task $\rightarrow$ target task.
- Same non-input operator across frames: FIFO edges from frame f to $f + 1$.

Evaluator Step 2: Ready-List Event Loop

The simulator maintains pending dependencies, earliest ready times, and one serialized ready time per resource:

$$time\_dev\_ready, \quad time\_host\_ready, \quad time\_network\_ready$$

A ready task starts at:

$$S_t = \begin{cases} ready_t, & resource(t) = source \\ \max(ready_t, time\_dev\_ready), & resource(t) = device \\ \max(ready_t, time\_host\_ready), & resource(t) = host \\ \max(ready_t, time\_network\_ready), & resource(t) = network \end{cases}$$

Main loop:

1. Put zero-pending tasks into the ready set.
2. Pick the ready task with the earliest feasible start time.
3. Break ties with the active rule priority.
4. Record start/finish, update the resource queue, and release successors.

Evaluator Step 3: Ensemble Rules

The task DAG and upward ranks are built once per assignment:

$$rank(t) = duration(t) + \max_{u \in succ(t)} rank(u)$$

The evaluator simulates three deterministic rules:

- **CriticalPath**: higher rank first, with older-frame aging

$$rank(t) + 10^6 \max(0, unroll - 1 - frame(t)).$$

- **DeviceFirst**: source tasks, then Device, then Network, then Host; duration and rank break ties.
- **FifoReady**: stable creation order after earliest feasible start time.

The earliest-start selection keeps the scheduler work-conserving; priorities decide only among tasks that can start at the same time.

Evaluator Step 4: Tail-Latency Retiming

The evaluator reduces artificial long tail-frame E2E windows without adding frame-wide gates:

- **Deferred blocked transfers**: if a cross-device target is still blocked by other prerequisites, delay the transfer until those prerequisites finish.
- **Right-shift sweep**: after list scheduling, move slack network tasks and join-side branch work later, as long as successors, resource order, releases, and output finish times remain unchanged.

This avoids counting early work that cannot help output completion as the frame's first non-input event, while preserving true inter-frame overlap.

Per assignment, the native core evaluates:

$$2 \times 3 = 6$$

deterministic list schedules: two transfer timings and three ready-list rules. Each schedule is then retimed by the conservative sweep before metric extraction.

Evaluator Step 5: Metrics and Winner Selection

Latency excludes source/input events. For each frame:

$$frame\_start = \min(\text{non-input compute starts, transfer starts})$$

$$frame\_finish = \max(\text{non-input output finishes})$$

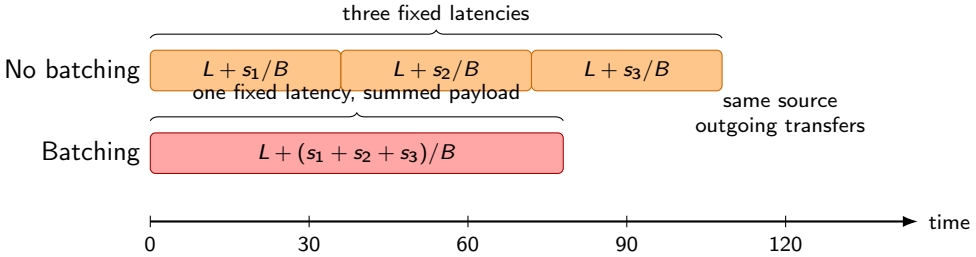
$$\tau_{frame} = frame\_finish - frame\_start$$

Reported metrics:

$$\tau_{max} = \max_{f} \tau_{frame}(f), \quad \tau_{avg} = \frac{pipeline\_finish - pipeline\_start}{pipeline\_unroll}$$

The winner is the schedule with lowest split loss. Ties prefer lower max-frame latency, then lower average latency, then higher Device utilization.

Network Batching Illustration



Batching reduces repeated fixed latency, but it does not allow network transfers to overlap.

Network Model: Serialized Transfers and Batching

- Cross-device transfers share one network worker timeline:

$$time\_network\_ready$$

- A transfer can start only after its source node finishes and the network worker is idle.
- Without batching, each cross-device edge is scheduled independently:

$$T_{edge} = L + \frac{s}{B}$$

- With `batch_transfers=true`, multiple outgoing cross-device edges from the same source node are sent as one transaction:

$$T_{batch} = L + \frac{\sum_k s_k}{B}$$

- This preserves non-overlap while avoiding repeated fixed latency for successive outgoing transfers.

Objective and Feasibility

The solver minimizes a weighted objective:

$$J = w_{avg} L_{avg} + w_{max} L_{max} + w_{util} L_{util}$$

where

$$L_{avg} = \frac{\tau}{\max(scale_{avg}, 1)}, \quad L_{max} = \frac{\tau^{max}_{frame}}{\max(scale_{max}, 1)}, \quad L_{util} = 1 - \frac{device\_active\_time}{pipeline\_work\_span}$$

The latency scales come from all-device/all-host baselines.

Latency limits

Positive limits act as hard feasibility constraints:

$$\tau \leq latency\_limit, \quad \tau^{max}_{frame} \leq max\_frame\_latency\_limit$$

A zero value disables that limit. Candidates above either enabled limit are rejected before objective comparison.

Solver Modes

Mode	Use case	Behavior
Auto	Default	Enumerates if ≤ 12 free nodes; otherwise random search
Enumerate	Small DAGs	Tests every assignment and returns the feasible global best
Random Search	Larger DAGs	Perturbs placements and keeps the best feasible candidate
Simulated Annealing	Escaping local minima	Accepts some worse moves according to temperature

All modes enforce fixed-device nodes plus the average E2E and max-frame latency caps.

GUI Workflow

1. Edit nodes, names, compute costs, fixed-device flags.
2. Edit edges and transfer sizes in MB.
3. Configure bandwidth, latency, network batching, objective weights, solver mode, iterations, and latency caps.
4. Click **Solve**.
5. Inspect metrics, topology placement, and profiler-style timeline.

Validation catches duplicate node IDs, unknown edge endpoints, and non-DAG topologies.

Visualization Design

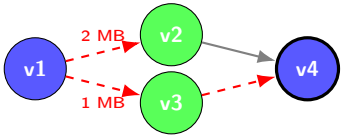
Topology SVG

- Config DAG shows transfer size and local/cross-device edge style before solving.
- Solved pipeline uses high-contrast node fill color for frame identity.
- Blue outline: DEVICE; green outline: HOST.
- Bold or dashed source styling marks fixed-device and input/source events.
- FIFO, transfer, local, and release lines follow the same frame color as the source frame.

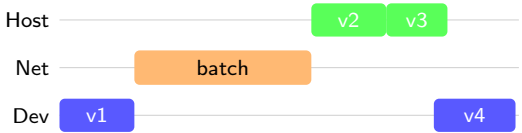
Timeline SVG

- Input, Host, Transfer, and Device lanes.
- Operator bars use computed start/finish times and the same frame fill color as the solved topology.
- Transfer bars use actual evaluator records, including serialized and batched transfers.
- Input release dots, spans, and vertical release/dependency lines use the matching frame color.
- Browser-native scroll and scale controls.

Topology and Timeline Reading Guide

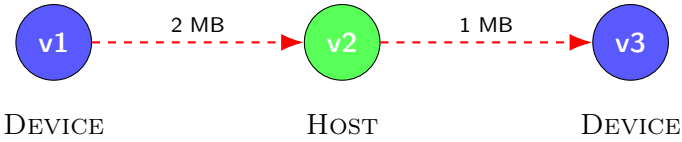


Topology: colors show placement; edge style shows communication



Timeline: bars show actual evaluator start/finish times

Example DAG



The evaluator produces a queue-aware schedule rather than an ideal infinite-parallel critical path.

Current Implementation Stack

- Python 3.9+.
- PyQt6 for desktop controls and layout.
- PyQt6-WebEngine for SVG topology and timeline rendering.
- NetworkX for DAG storage, topological sort, and validation.
- C++/pybind11 native core for schedule evaluation, objective scoring, and solver search.
- Pytest coverage for evaluator, JSON IO, solver modes, and latency-limit feasibility.
- Launch commands: `python -m coopinfer` or `python -m coopinfer.app`.

Next Steps

- Add richer solver diagnostics: rejected candidates, constraint slack, convergence history.
- Add exportable SVG/PNG reports for topology and timeline.
- Add batch evaluation over bandwidth/latency sweeps.
- Expose selected scheduling rule, queue slack, and ready-task diagnostics.
- Improve timeline semantics with explicit queue-wait and data-ready intervals.
- Explore additional network reordering policies and transfer-priority rules.

Questions?